

Quokka Express

Solution description | Thomas Zeng (24181084), Group 91

Problem summary

Quincy the quokka starts at camp 1 and needs to reach camp n . The trails go both ways and have different walking times. Quincy has one pass that can make one trail take 0 minutes. We need to find the smallest total time.

Main idea: two states for each camp

This uses Dijkstra's algorithm from the CITS3001 shortest path lectures. We need to know if the pass has been used. Being at a camp with the pass is not the same as being there without it. Every camp v has two states:

State	Meaning
$(v, 0)$	At camp v ; the pass is ready
$(v, 1)$	At camp v ; the pass was used

Store the trails in an adjacency list and keep states in a binary min-heap. For a trail (u, v) with time w , we can move normally and add w . If the pass is ready, we can also move for free and change from state 0 to state 1. Run Dijkstra from $(1, 0)$. The answer is $\min(d[n][0], d[n][1])$.

Pseudocode

```
dist[1][0] = 0; push (0, 1, 0)
while the heap is not empty:
    take (d, u, used) from the heap; skip old entries
    for each trail (u, v, w): try cost d + w for (v, used)
    if used = 0: also try cost d for (v, 1)
return min(dist[n][0], dist[n][1])
```

Why it works

- 1. Every real trip makes a state path.** Before Quincy uses the pass, he stays in state 0. When he uses it, he moves to state 1 and pays 0 for that trail. After this, he stays in state 1. The state path has the same total time.
- 2. Every state path follows the rules.** A path ending in state 0 did not use the pass. A path ending in state 1 changed from 0 to 1 one time. It cannot go back to 0, so the pass cannot be used twice.
- 3. We get the best answer.** Every move costs 0 or more, so Dijkstra finds the shortest path to both ending states. By steps 1 and 2, the smaller distance is the smallest time Quincy needs.

Complexity

There are **$2n$ states** and $O(m)$ state moves. Each heap operation costs $O(\log n)$, so the time is **$O((n + m) \log n)$** . The adjacency list and distances use **$O(n + m)$ space**.

The answer can be very large, so the infinity value must also be very large. Python integers already have enough range.

Why simple ideas fail

One wrong idea is to find the normal shortest path, then use the pass on its biggest trail. The pass may make a different path better. Trying every trail and running Dijkstra again takes $O(m(n + m) \log n)$ time, so it is too slow.

AI use

I used ChatGPT/Codex to help me plan the story, write and shorten the English, make test ideas, write and check the Python solutions, and put the problem files together. I read the final problem, code, tests, and solution notes and checked that they work.